



1. AI-Powered Code Reviewer

1. Problem Statement

Junior developers push buggy and unoptimised code because peer reviews take days and senior engineers are bottlenecked. There is no scalable tool that can instantly review any code snippet, catch bugs, flag security vulnerabilities, and suggest improvements with clear explanations.

2. Solution

- User pastes code or uploads a file which is sent to GPT-4o with a structured system prompt requesting JSON output covering bugs, security issues, performance suggestions, and a cleaned rewrite.
- Results are rendered in a side-by-side diff view within a Flask web application so developers can compare the original and improved code instantly.

3. Key Features

- Paste or upload code in any language — instant review in seconds
- Side-by-side original vs suggested rewrite with diff highlighting
- Severity tags per issue: Critical / Warning / Suggestion
- Copy improved code button — one click to clipboard

4. Technology Stack

- Python — prompt builder, response parser, Flask application server
- Flask — REST backend with /review endpoint handling code input and GPT response
- OpenAI GPT-4o API — structured JSON output mode for categorised review results
- HTML + CSS — diff viewer layout, severity badge styling, copy button
- JavaScript — async fetch to Flask API, dynamic diff rendering and clipboard handler

5. Expected Outcomes / Benefits

- Junior developers get instant, structured feedback without waiting for senior review
- Code quality improves consistently across teams with zero reviewer bottleneck
- Security issues caught before they reach production
- Scalable to any language or codebase with a single prompt adjustment

Dataset: CodeSearchNet (GitHub code dataset) | **Link:** <https://www.kaggle.com/datasets/omdena/codesearchnet>





2. Local Document Q&A Chatbot

1. Problem Statement

Companies have hundreds of internal PDFs, policy documents, and manuals. Employees waste hours searching manually and cannot query document contents in natural language. Sending sensitive internal documents to external APIs is a data privacy and compliance risk.

2. Solution

- Chunk and embed uploaded documents using sentence-transformers, store vector representations in ChromaDB, and retrieve the most relevant chunks when a user asks a question.
- Pass retrieved chunks to Mistral 7B running locally via Ollama for answer generation — the entire pipeline runs offline with no data leaving the machine.

3. Key Features

- Upload any PDF — chunked and indexed instantly with source tracking
- Natural language Q&A with source page citation per answer
- Fully offline — no data leaves the machine, no API key required
- Chat history with context-aware follow-up questions

4. Technology Stack

- Python — document chunking, embedding pipeline, query orchestration
- Flask — web server handling file upload and chat API endpoints
- Ollama + Mistral 7B — local LLM inference, no external API
- sentence-transformers (all-MiniLM-L6-v2) — text vectorisation for semantic search
- ChromaDB — vector store for document chunk retrieval
- HTML + CSS + JavaScript — chat interface with file upload and source citation display

5. Expected Outcomes / Benefits

- Employees can query internal documents in seconds without manual search
- Sensitive data stays on-premise — full compliance with data privacy requirements
- Zero API cost — runs entirely on local hardware
- Replicable for any organisation with PDF-heavy knowledge bases

Dataset: Any internal PDF corpus — demo with Wikipedia or ArXiv papers | **Link:**

<https://www.kaggle.com/datasets/Cornell-University/arxiv>





3. Custom Image Classifier Web App

1. Problem Statement

Hospitals, factories, and retail businesses need to classify images specific to their domain — tumour vs healthy tissue, defective vs acceptable products, real vs counterfeit items. Generic pre-trained APIs cannot learn domain-specific visual patterns without custom training.

2. Solution

- Collect a labelled image dataset, build and train a Convolutional Neural Network in PyTorch with data augmentation to improve generalisation.
- Evaluate on a held-out test set and deploy via Flask so non-technical users can upload an image through a web form and receive a predicted class with a confidence score.

3. Key Features

- Image upload form — predicts class with confidence percentage instantly
- Training pipeline with accuracy and loss curve visualisation
- Model retraining endpoint when new labelled images are added
- Confusion matrix report displayed on the model performance page

4. Technology Stack

- Python — dataset loading, CNN architecture definition, training loop
- PyTorch — CNN model built and trained from scratch with custom architecture
- torchvision — data augmentation transforms (flip, rotate, normalise)
- Flask — /predict inference endpoint and /retrain endpoint
- HTML + CSS + JavaScript — image upload form, prediction result card, training curve chart

5. Expected Outcomes / Benefits

- Domain-specific classification that generic APIs cannot provide
- Non-technical users get a browser-based tool requiring no ML knowledge
- Model improves continuously as new labelled images are added and retraining is triggered
- Framework reusable for any image classification problem by swapping the dataset

Dataset: CIFAR-10 / Intel Image Classification Dataset | **Link:** <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>





4. AI Customer Support Agent

1. Problem Statement

E-commerce support teams are overwhelmed by repetitive Tier-1 queries — order status, return policy, refund requests. Manual handling is slow and expensive, but fully generic AI responses without intent understanding produce irrelevant or incorrect answers.

2. Solution

- Fine-tune a small BERT model on labelled support queries to classify intent — order, refund, complaint, or other — with high accuracy and low latency.
- Route classified intents to GPT-4o with a domain-specific system prompt for response generation, and flag low-confidence queries for human escalation.

3. Key Features

- Natural language query input — instant AI response with intent label
- Intent classifier routes between template responses and full GPT-4o generation
- Escalation flag for low-confidence or sensitive queries sent to human queue
- Admin dashboard showing query volume, intent breakdown, and escalation rate

4. Technology Stack

- Python — BERT fine-tuning pipeline, intent routing logic, query logging
- Hugging Face Transformers — BERT fine-tuning on labelled support query dataset
- OpenAI GPT-4o API — response generation with domain-specific system prompt
- Flask — /query endpoint, routing logic, admin report endpoints
- HTML + CSS + JavaScript — customer chat widget and admin analytics dashboard

5. Expected Outcomes / Benefits

- Tier-1 queries handled automatically — support team focuses on complex issues only
- Faster response time for customers — seconds instead of hours
- Cost reduction in customer support headcount for repetitive query handling
- Admin dashboard provides visibility into query patterns for continuous improvement

Dataset: Bitext Customer Support Intent Dataset | **Link:** <https://www.kaggle.com/datasets/bitext/training-dataset-for-chatbots-virtual-assistants>





5. Real-Time Speech-to-Summary Tool

1. Problem Statement

Students, journalists, and analysts sit through long meetings, lectures, and interviews and then spend equal time writing summaries manually. No affordable offline tool exists that both transcribes audio and produces a structured summary automatically.

2. Solution

- Run OpenAI Whisper locally for high-accuracy offline speech-to-text transcription of uploaded audio files.
- Pass the full transcript to facebook/bart-large-cnn on Hugging Face for abstractive summarisation, producing a clean structured summary with key bullet points — all without sending data to any external API.

3. Key Features

- Upload audio file (MP3/WAV) or record live from browser microphone
- Full transcript with timestamped segments for audio navigation
- Structured summary with key bullet points extracted by BART
- Export transcript and summary as DOCX or PDF

4. Technology Stack

- Python — audio preprocessing, Whisper transcription pipeline, BART summarisation
- openai-whisper — local offline speech recognition model
- Hugging Face Transformers — facebook/bart-large-cnn abstractive summarisation
- Flask — audio upload endpoint, transcription and summary API routes
- HTML + CSS + JavaScript — audio recorder, upload form, and structured output display

5. Expected Outcomes / Benefits

- Hours of manual note-taking eliminated for students and professionals
- Fully offline — suitable for confidential meetings and interviews
- Zero subscription cost compared to commercial transcription services
- Exportable output integrates directly into existing workflows

Dataset: LibriSpeech ASR Corpus / TED-LIUM | **Link:** <https://www.kaggle.com/datasets/pavanelisetty/sample-audio-files-for-speech-recognition>





6. Sentiment Analysis API for Product Reviews

1. Problem Statement

Brands monitor thousands of product reviews daily but have no automated pipeline to track sentiment trends over time. Manual analysis is impossible at scale, and off-the-shelf tools do not expose a trainable API that teams can integrate into their own dashboards.

2. Solution

- Preprocess a labelled Amazon review dataset, build and train an LSTM neural network in PyTorch achieving above 88% accuracy, and serialise the trained model.
- Wrap the model in a Flask REST API with single-prediction and batch endpoints, and build a demo dashboard showing sentiment trend charts over uploaded review batches.

3. Key Features

- REST API endpoint: POST review text → JSON response with sentiment and confidence score
- Batch mode: upload CSV of reviews and download results with sentiment column appended
- Sentiment trend chart for uploaded review batches on the demo dashboard
- Model card page showing accuracy, precision, recall, and confusion matrix on test set

4. Technology Stack

- Python — text preprocessing, LSTM training loop, model serialisation with joblib
- PyTorch — LSTM architecture trained on labelled Amazon review dataset
- NLTK — tokenisation, stopword removal, and text normalisation
- Flask — /predict and /batch REST endpoints
- HTML + CSS + JavaScript — demo UI, CSV upload handler, Chart.js sentiment trend chart

5. Expected Outcomes / Benefits

- Marketing teams get real-time sentiment visibility without manual reading
- REST API plugs into any existing BI dashboard or analytics pipeline
- Model card provides transparency for stakeholders reviewing model reliability
- Retractable on any domain-specific review data by replacing the training dataset

Dataset: Amazon Product Reviews (Multi-Category) | **Link:**

<https://www.kaggle.com/datasets/cynthiarempel/amazon-us-customer-reviews-dataset>





7. AI Study Assistant for Students

1. Problem Statement

Students upload lecture PDFs or paste notes and need help understanding complex concepts, generating practice quiz questions, and preparing for exams. No affordable personalised study tool exists that adapts to any subject without subject-specific training.

2. Solution

- Use Gemini API with four study modes powered by mode-specific system prompts: Explain (simplify a concept), Quiz Me (generate MCQs), Summarise (bullet-point notes), and Exam Prep (predict likely exam questions).
- PDF text is extracted with PyMuPDF, passed to Gemini with the selected mode prompt, and results are rendered in a tabbed Flask web interface.

3. Key Features

- Four study modes: Explain, Quiz Me, Summarise, and Exam Prep
- PDF upload or paste notes directly — works for any subject
- Interactive MCQ quiz mode with instant answer reveal and running score
- Export quiz results and summaries as PDF for offline revision

4. Technology Stack

- Python — mode routing logic, PDF text extraction, Gemini prompt construction
- Google Gemini API (gemini-1.5-flash) — multimodal text and document understanding
- PyMuPDF — extract clean text from uploaded lecture PDFs
- Flask — session management and mode-specific API routes
- HTML + CSS + JavaScript — tabbed study interface, quiz reveal logic, score tracker

5. Expected Outcomes / Benefits

- Students get instant, subject-agnostic study support without tutors or subscriptions
- Quiz Me mode improves active recall — proven to increase retention vs passive reading
- Exam Prep mode surfaces likely questions, reducing exam anxiety
- Adaptable to any subject or course without any model retraining

Dataset: Open University Learning Analytics Dataset (OULAD) | **Link:**

<https://www.kaggle.com/datasets/anlgrbz/student-demographics-online-education-dataoulad>





8. AI-Generated Image Detector

1. Problem Statement

AI-generated deepfakes and synthetic images are spreading across news, social media, legal proceedings, and HR verification workflows. There is no accessible web tool that can classify whether an image is real or AI-generated with a visual explanation of the decision.

2. Solution

- Fine-tune ResNet-50 on the CIFAKE dataset of real vs AI-generated images using PyTorch, and optionally use CLIP embeddings as additional features for a secondary classification layer.
- Deploy via Flask so users upload an image and receive a Real / AI-Generated verdict with a confidence score and a Grad-CAM heatmap highlighting which regions influenced the decision.

3. Key Features

- Upload image — Real or AI-Generated verdict with confidence percentage
- Grad-CAM heatmap showing which regions of the image triggered the classification
- Batch mode: upload multiple images and download a results CSV
- Model stats page with accuracy, ROC curve, and precision-recall on the test set

4. Technology Stack

- Python — image preprocessing, ResNet-50 fine-tuning, Grad-CAM heatmap generation
- PyTorch — ResNet-50 fine-tuned on CIFAKE real vs AI-generated dataset
- CLIP (open-source, Hugging Face) — semantic image embedding for secondary features
- Flask — /predict inference endpoint and /batch endpoint
- HTML + CSS + JavaScript — image upload form, verdict card, heatmap overlay display

5. Expected Outcomes / Benefits

- Journalists and fact-checkers can verify image authenticity in seconds
- HR and legal teams gain a tool for detecting AI-fabricated identity documents
- Visual heatmap explanation builds user trust in the model verdict
- Batch mode enables platform-level moderation at scale

Dataset: CIFAKE — Real and AI-Generated Synthetic Images | **Link:**

<https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images>





9. Multilingual Content Moderator

1. Problem Statement

Online platforms with global users receive harmful, toxic, and spam content in dozens of languages simultaneously. English-only moderation tools miss the majority of violations. Building separate classifiers per language requires expensive labelled data for each language.

2. Solution

- Use XLM-RoBERTa with Hugging Face zero-shot classification pipeline to classify posts as toxic, non-toxic, or spam across 100+ languages without requiring any per-language labelled training data.
- Build a moderation queue dashboard where flagged posts are sorted by confidence score for human review, with bulk approve/reject actions and analytics on flagged content trends.

3. Key Features

- Classify text in 100+ languages as Toxic / Non-Toxic / Spam with confidence score
- Zero-shot approach — no labelled training data required per language
- Moderation queue: flagged posts sorted by confidence for human reviewer action
- Analytics dashboard showing flagged volume by language and category over time

4. Technology Stack

- Python — Hugging Face pipeline setup, batch classification, queue management logic
- Hugging Face Transformers — XLM-RoBERTa zero-shot classification pipeline
- Flask — /classify and /queue API endpoints
- SQLite — moderation queue storage and action log
- HTML + CSS + JavaScript — moderator dashboard, bulk action controls, Chart.js analytics

5. Expected Outcomes / Benefits

- Toxic content detected across all platform languages without language-specific training
- Moderation team workload reduced — only borderline cases need human review
- Analytics reveal which languages and topics generate the most violations
- Zero labelled data cost — deployable immediately on any language mix

Dataset: Jigsaw Multilingual Toxic Comment Classification | **Link:** <https://www.kaggle.com/competitions/jigsaw-multilingual-toxic-comment-classification>





10. Stock Price Movement Predictor

1. Problem Statement

Retail investors and finance students need a hands-on project combining real financial data with deep learning. No affordable accessible tool exists that trains a custom prediction model on live stock data and presents results in a clean, interpretable web interface.

2. Solution

- Fetch historical OHLCV stock data via yfinance, engineer technical indicators — RSI, MACD, Bollinger Bands — as model features, and train an LSTM on sliding windows of price history to predict next-day direction.
- Deploy a Flask web app where users enter any stock ticker, the model fetches live data, runs inference, and displays the next-day forecast with a candlestick chart and indicator overlays.

3. Key Features

- Enter any stock ticker — model fetches live data via yfinance and runs inference
- Next-day direction forecast: Up or Down with probability score
- Candlestick chart with RSI, MACD, and Bollinger Band overlays on the results page
- Backtesting report showing model accuracy and simulated return vs buy-and-hold

4. Technology Stack

- Python — OHLCV data fetch via yfinance, technical indicator engineering, LSTM training
- PyTorch — LSTM model trained on sliding window of price and indicator features
- yfinance — live historical stock data retrieval for any ticker
- Flask — /predict endpoint handling ticker input, data fetch, and inference
- HTML + CSS + JavaScript — ticker input form, Chart.js candlestick chart, forecast card

5. Expected Outcomes / Benefits

- Finance students build a complete end-to-end ML pipeline on real market data
- Demonstrates LSTM time series modelling — a highly valued skill in fintech roles
- Live data integration via yfinance keeps the app relevant and continuously testable
- Backtesting report provides rigorous model evaluation beyond simple accuracy metrics

Dataset: S&P 500 Stock Data / Yahoo Finance via yfinance |

Link: <https://www.kaggle.com/datasets/camnugent/sandp500>

